

Spis treści

Spis treści	1
1. Opis projektu	1
2. Analiza Interesariuszy (User Roles)	1
3. Opracowanie Wymagań Funkcjonalnych	1
4. Scenariusze User Stories	2
5. Opracowanie Wymagań Niefunkcjonalnych	3
6. Modelowanie Procesu (Workflow)	3
7. Diagram Sekwencji (Sequence Diagram)	4
8. Diagram Przepływu Stanów Dokumentu (State Diagram)	5
9. Diagram Przepływu (Flowchart) - Logika Uprawnień	7
10. Modelowanie wszystkich możliwych scenariuszy	9
11. Opracowanie schematu ERD Bazy Danych	14
12. Opracowanie szkieletu tabel w SQL	16
13. Dobór systemu bazy danych	18
14. Dobór narzędzia do prototypowania bazy danych	18
15. Propozycja narzędzia do organizacji projektu bazy danych	19
16. Opracowanie diagramu ERD dla formularza praktyki oraz wskazanie kluczy głównych i obcych	19
17. Rozszerzony model ERD (Postgres)	23

1. Opis projektu

Projekt dotyczy budowy aplikacji internetowej wspomagającej obieg dokumentacji praktyk zawodowych. System ma zastąpić tradycyjny, papierowy obieg dokumentów (dziennik praktyk, arkusz efektów, uczenia się, opinie opiekunów) procesem cyfrowym z możliwością generowania PDF.

2. Analiza Interesariuszy (User Roles)

Na podstawie opisu systemu zidentyfikowano następujących kluczowych interesariuszy oraz przypisano cele:

- **Student:** tworzy, uzupełnia i przesyła dokumentację praktyk do zatwierdzenia; śledzi postęp realizacji efektów uczenia się i uzyskuje zaliczenie.
- **Opiekun Zakładowy (ZOPZ):** weryfikuje i zatwierdza dziennik praktyk, potwierdza osiągnięcie efektów, wystawia ocenę zakładową (opisową i parametryczną) oraz potwierdza sprawozdanie.
- **Opiekun Uczelniany (UOPZ):** weryfikuje dziennik i opinie ZOPZ, wystawia ocenę uczelnianą i ocenę sprawozdania, a na końcu podpisuje protokół i finalizuje zaliczenie (wpis do USOS).
- **Dyrektor Instytutu:** odpowiada za formalności: porozumienia (Zał. 1), kierowanie na praktykę (Zał. 3.1) oraz wystawienie Protokołu Zaliczenia (Zał. 8) jako Przewodniczący Komisji.
- **Administrator systemu:** zarządza kontami użytkowników (zatwierdza rejestracje i nadaje role), rejestruje praktyki i firmy oraz nadzoruje archiwizację dokumentów.

3. Opracowanie Wymagań Funkcjonalnych

Wymagania funkcjonalne opisują, co system ma robić, stworzono tabelaryczne zestawienie:

ID	Nazwa	Opis	Priorytet MoSCoW
1	Wypełnianie dziennika	System umożliwia Studentowi na wypełnianie Dziennika Praktyk w formie dynamicznej tabeli (dodawanie daty, opisu prac i efektów uczenia się).	Must
2	Przesyłanie do weryfikacji	System umożliwia Studentowi na przesłanie dokumentu do weryfikacji, co uruchamia mechanizm automatycznego blokowania dalszej edycji przez studenta.	Must

3	Dodawanie uwag	System umożliwia Opiekunom odrzucenie dokumentu z podaniem uzasadnienia oraz uzupełnienie opinii opisowej, co odblokowuje dokument do poprawy przez Studenta.	Must
4	Potwierdzanie zadań	System umożliwia Opiekunowi Zakładowemu na elektroniczne potwierdzanie wykonania poszczególnych prac opisanych w Dzienniku Praktyk.	Must
5	Wystawianie oceny	System umożliwia wystawienie ocen częściowych przez opiekunów oraz wygenerowanie końcowego Protokołu Zaliczenia (ocena K) przez Dyrektora; UOPZ finalizuje zaliczenie.	Must
6	Wystawienie opinii zakładowej	System umożliwia Opiekunowi Zakładowemu na uzupełnienie oceny opisowej oraz parametrycznej z przebiegu praktyki.	Must
7	Raport PDF	System umożliwia Studentowi i Opiekunom na generowanie pełnego raportu z przebiegu praktyki w pliku PDF (zawierającego m.in. dziennik i opinie).	Must
8	Zatwierdzanie kont spoza organizacji uczelni	System umożliwia Administratorowi zatwierdzanie kont i nadawanie ról	Must

4. Scenariusze User Stories

Wybrano 3 kluczowe funkcjonalności i opisane je w formacie *User Story*: „Jako [rola], chcę [cel], aby [korzyść]”:

- Jako **Student**, chcę uzupełniać dziennik praktyk w dynamicznej tabeli z walidacją na bieżąco (data w okresie praktyki, tylko dni robocze, powiązanie wpisu z efektami uczenia się), aby mieć pewność, że dziennik jest kompletny i poprawny, zanim trafi do zatwierdzenia przez opiekuna
- Jako **Opiekun Zakładowy**, chcę móc zatwierdzić dziennik dopiero, gdy zawiera komplet 120 dni roboczych i pokrywa wszystkie 13 efektów uczenia się, aby mieć gwarancję, że student zrealizował pełny program praktyki przed wystawieniem oceny zakładowej.
- Jako **Opiekun Uczelniany**, chcę po sfinalizowaniu zaliczenia wygenerować komplet dokumentów w formacie PDF (pojedynczo lub jako archiwum ZIP wszystkich załączników i dziennika), aby uzyskać cyfrową wersję dokumentacji do archiwizacji w aktach studenta.

5. Opracowanie Wymagań Niefunkcjonalnych

Zdefiniowano wymagania niefunkcjonalne (jakościowe) dla aplikacji, biorąc głównie pod uwagę następujące kategorie:

- **Bezpieczeństwo:** Dostęp do elektronicznego Dziennika Praktyk, szczegółowych danych o przebiegu stażu oraz profili interesariuszy jest restrykcyjny - dane konkretnego studenta mogą być przeglądane i modyfikowane tylko przez autoryzowanych (ten konkretny student oraz przypisani do niego Opiekunowie i Sekretariat).
- **Użyteczność:** Interfejs dynamicznej tabeli w Dzienniku Praktyk musi być w pełni responsywny i czytelny na urządzeniach mobilnych, aby student mógł na bieżąco, np. na smartfonie, dodawać opisy wykonanych w danym dniu prac.
- **Wydajność:** Mechanizm systemu odpowiadający za łączenie wszystkich arkuszy i opinii powinien generować końcowy raport w formacie PDF w czasie nie dłuższym niż 5 sekund od momentu kliknięcia przycisku przez użytkownika.
- **Archiwizacja:** Zgromadzone przez system dane (wygenerowane PDF-y, wnioski o uznanie kompetencji z pracy zawodowej, elektroniczne dzienniki) są trwale przechowywane w systemie bazy danych ze stałą kopią zapasową w formacie chroniącym integralność dokumentacji przed jej utratą.

6. Modelowanie Procesu (Workflow)

Przedstawiono cykl życia aplikacji w formie listy kroków (od momentu utworzenia przez studenta, poprzez weryfikację i poprawki, aż do zatwierdzenia przez opiekuna uczelnianego):

1. **Faza 1: Inicjacja.** Dyrektor rejestruje praktykę (dane firmy, ramy czasowe). Powstają i są podpisywane dokumenty wstępne: Porozumienie (Zał. 1), Harmonogram (Zał. 2a), Skierowanie i potwierdzenia zakładu, zgłoszenie + BHP (Zał. 3.1). Po komplecie podpisów proces przechodzi do Fazy 2.
2. **Faza 2: Realizacja.** Student systematycznie uzupełnia dynamiczny Dziennik Praktyk (120 dni roboczych), wiążąc każdy wpis z kodami efektów uczenia się (01–13). Po wypełnieniu przesyła dziennik; system waliduje komplet 120 dni i pokrycie wszystkich 13 efektów. ZOPZ zatwierdza dziennik, co blokuje dalszą edycję i przenosi proces do Fazy 3.
3. **Faza 3: Podsumowanie.** Uzupełniane i podpisywane są dokumenty końcowe: Karta praktyki str. 2 z ocenami zakładową i uczelnianą (Zał. 3.2), Potwierdzenie efektów (Zał. 4), Sprawozdanie studenta potwierdzone przez ZOPZ (Zał. 7) oraz Kwestionariusz ankiety (Zał. 5). Po komplecie proces przechodzi do Fazy 4.
4. **Faza 4: Zaliczenie.** Dyrektor (Przewodniczący Komisji) wystawia Protokół Zaliczenia z oceną końcową K (Zał. 8). UOPZ podpisuje protokół i finalizuje zaliczenie, praktyka zostaje oznaczona jako Zakończona. Na każdym etapie dostępne jest generowanie dokumentów do PDF (pojedynczo lub jako ZIP) do archiwizacji.

7. Diagram Sekwencji (Sequence Diagram)

Zaprojektowano diagram sekwencji przedstawiający proces weryfikacji Dziennika Praktyk. Uwzględniono interakcję między Studentem, Systemem, Bazą Postgres oraz Opiekunem Uczelnianym:

Kod Mermaid:

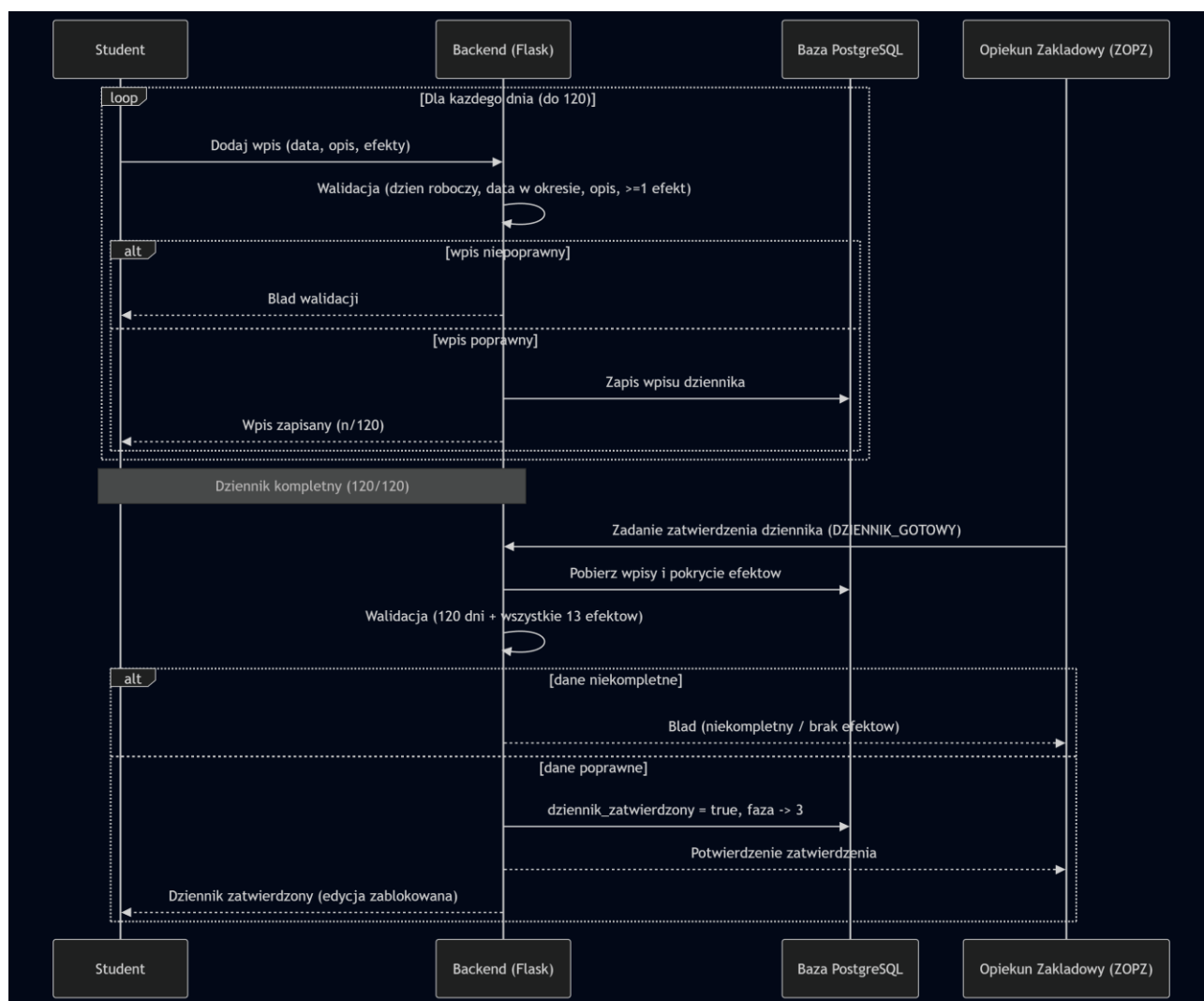
```
sequenceDiagram
    participant S as Student
    participant B as Backend (Flask)
    participant DB as Baza PostgreSQL
    participant Z as Opiekun Zakładowy (ZOPZ)

    loop Dla każdego dnia (do 120)
        S->>B: Dodaj wpis (data, opis, efekty)
        B->>B: Walidacja (dzień roboczy, data w okresie, opis, >=1 efekt)
        alt wpis niepoprawny
            B-->>S: Błąd walidacji
        else wpis poprawny
            B->>DB: Zapis wpisu dziennika
            B-->>S: Wpis zapisany (n/120)
        end
    end

    Note over S,B: Dziennik kompletny (120/120)

    Z->>B: Zadanie zatwierdzenia dziennika (DZIENNIK_GOTOWY)
    B->>DB: Pobierz wpisy i pokrycie efektów
    B->>B: Walidacja (120 dni + wszystkie 13 efektów)
    alt dane niekompletne
        B-->>Z: Błąd (niekompletny / brak efektów)
    else dane poprawne
        B->>DB: dziennik_zatwierdzony = true, faza -> 3
        B-->>Z: Potwierdzenie zatwierdzenia
        B-->>S: Dziennik zatwierdzony (edycja zablokowana)
    end
```

Wygenerowany z kodu Mermaid diagram przedstawiono na Rys. 1.:



Rys. 1. Diagram sekwencji Dziennika Praktyk

8. Diagram Przepływu Stanów Dokumentu (State Diagram)

Wykorzystano stateDiagram-v2 w Mermaid, aby zamodelować cykl życia wniosku o praktykę, uwzględniono następujące stany:

- Draft (szkic)
- Submitted (zgłoszono)
- Under_Review (w trakcie przeglądu)
- Rejected (odrzucono)
- Approved (zatwierdzono)
- Closed (zamknięto)
- Decision (warunki, gdy opiekun zgłosi uwagi)

Kod Mermaid:

```
stateDiagram-v2
    [*] --> Draft : Utworzenie praktyki (Faza 2)

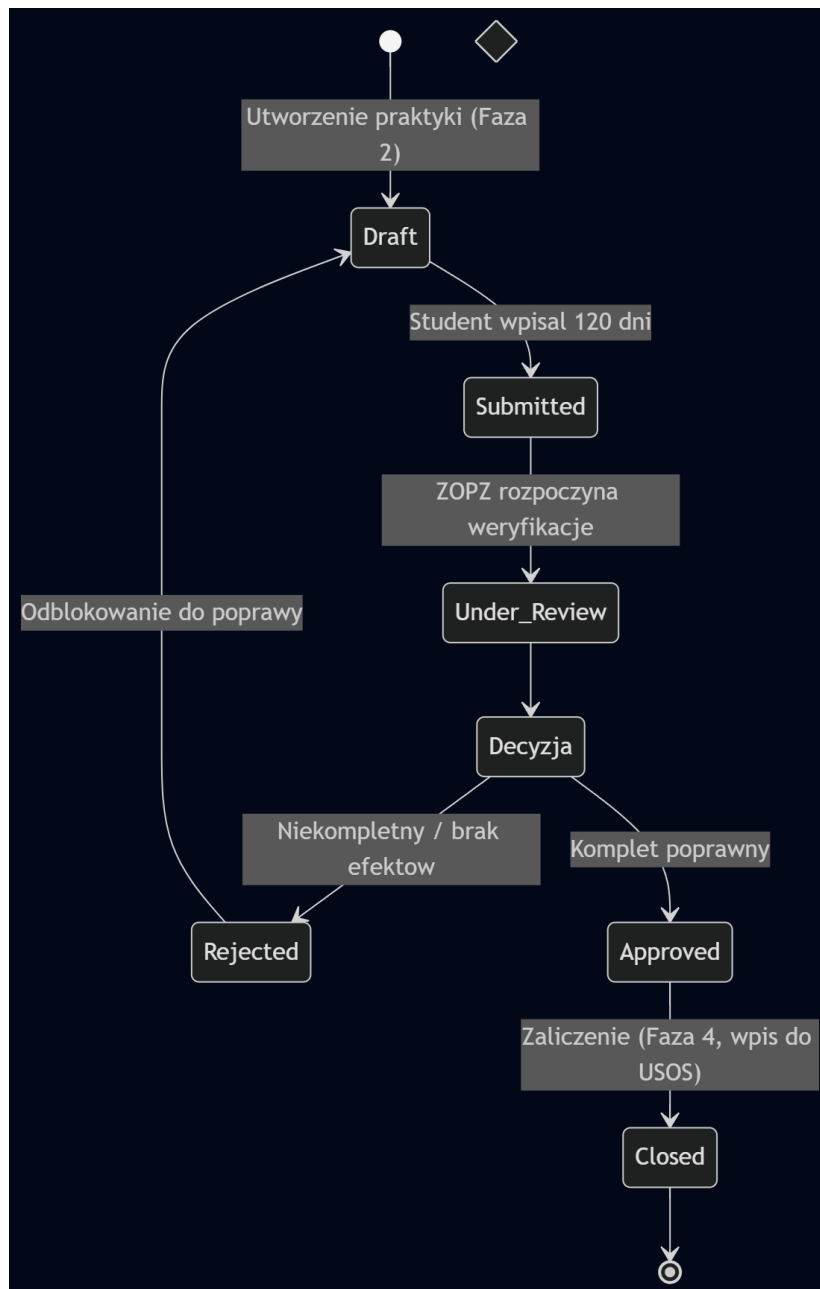
    Draft --> Submitted : Student wpisał 120 dni
    Submitted --> Under_Review : ZOPZ rozpoczyna weryfikacje

    state "Walidacja (120 dni + 13 efektow)" as Decyzja <<choice>>
    Under_Review --> Decyzja

    Decyzja --> Rejected : Niekompletny / brak efektow
    Decyzja --> Approved : Komplet poprawny

    Rejected --> Draft : Odblokowanie do poprawy
    Approved --> Closed : Zaliczenie (Faza 4, wpis do USOS)
    Closed --> [*]
```

Wygenerowany z kodu Mermaid diagram przedstawiono na Rys. 2.:



Rys. 2. Diagram Przepływu Stanów Dziennika Praktyk

9. Diagram Przepływu (Flowchart) - Logika Uprawnień

Zamodelowano algorytm, który system wykonuje podczas próby edycji dokumentu przez użytkownika.

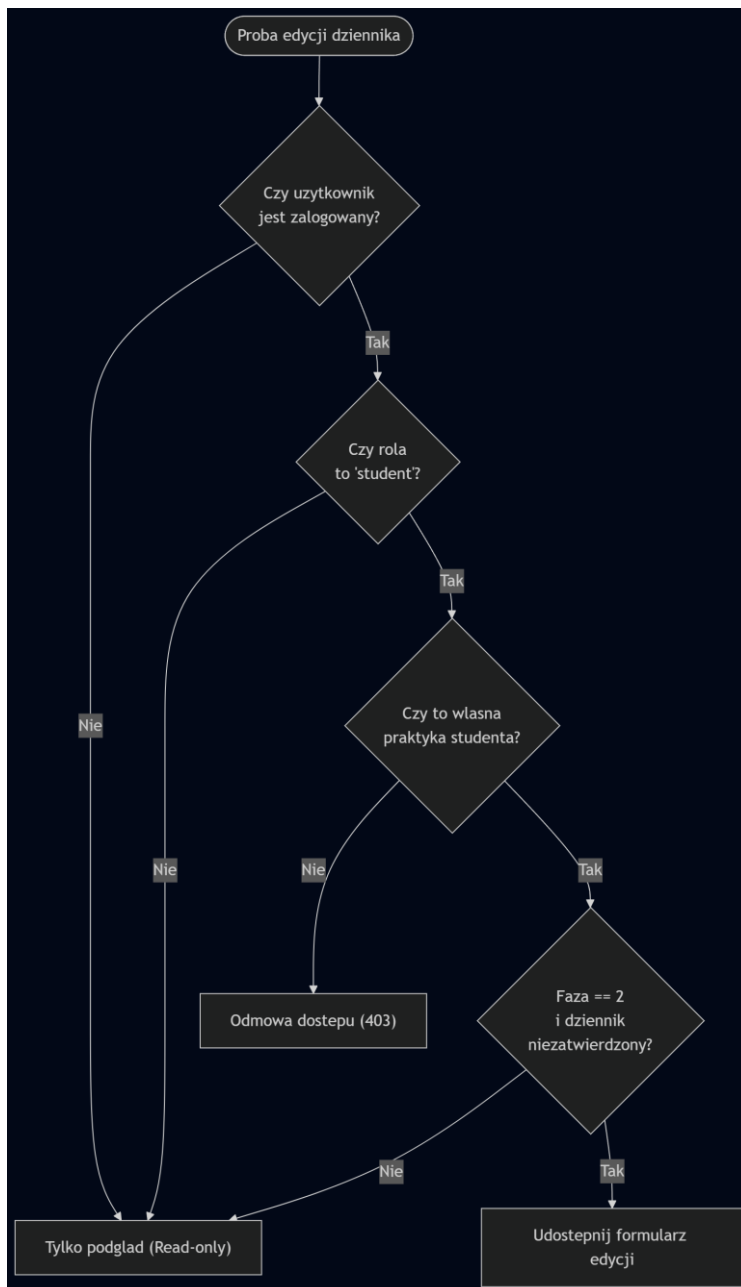
Uwzględniono warunki:

- Czy użytkownik jest zalogowany?
- Czy rola użytkownika to 'Student'?
- Czy status dokumentu to 'Draft' lub 'Rejected'?
- Jeśli tak → udostępni formularz edycji. Jeśli nie → wyświetl tylko podgląd (Read-only).

Kod Mermaid:

```
flowchart TD
    Start([Proba edycji dziennika]) --> Q1{Czy uzytkownik<br/>jest zalogowany?}
    Q1 -- Nie --> R0["Tylko podglad (Read-only)"]
    Q1 -- Tak --> Q2{Czy rola<br/>to 'student'?}
    Q2 -- Nie --> R0
    Q2 -- Tak --> Q3{Czy to wlasna<br/>praktyka studenta?}
    Q3 -- Nie --> B403["Odmowa dostepu (403)"]
    Q3 -- Tak --> Q4{Faza == 2<br/>i dziennik<br/>niezatwierdzony?}
    Q4 -- Nie --> R0
    Q4 -- Tak --> Edit[Edit["Udostepnij formularz edycji"]]
```

Wygenerowany z kodu Mermaid diagram przedstawiono na Rys. 3.:



Rys. 3. Flowchart prezentujący logikę uprawnień

10. Modelowanie wszystkich możliwych scenariuszy

W postaci diagramów przepływów zamodelowane następujące interakcje:

- Interakcje związane z interfejsem
- Interakcje związane z przepływem danych
- Interakcje związane z logiką biznesową - zakodowane mechanizmy pracy aplikacji

1. Interakcje związane z interfejsem: Ten diagram skupia się wyłącznie na tym, co widzi użytkownik i jak nawiguje po aplikacji (przejścia między ekranami, formularzami i komunikatami). Kod Mermaid:

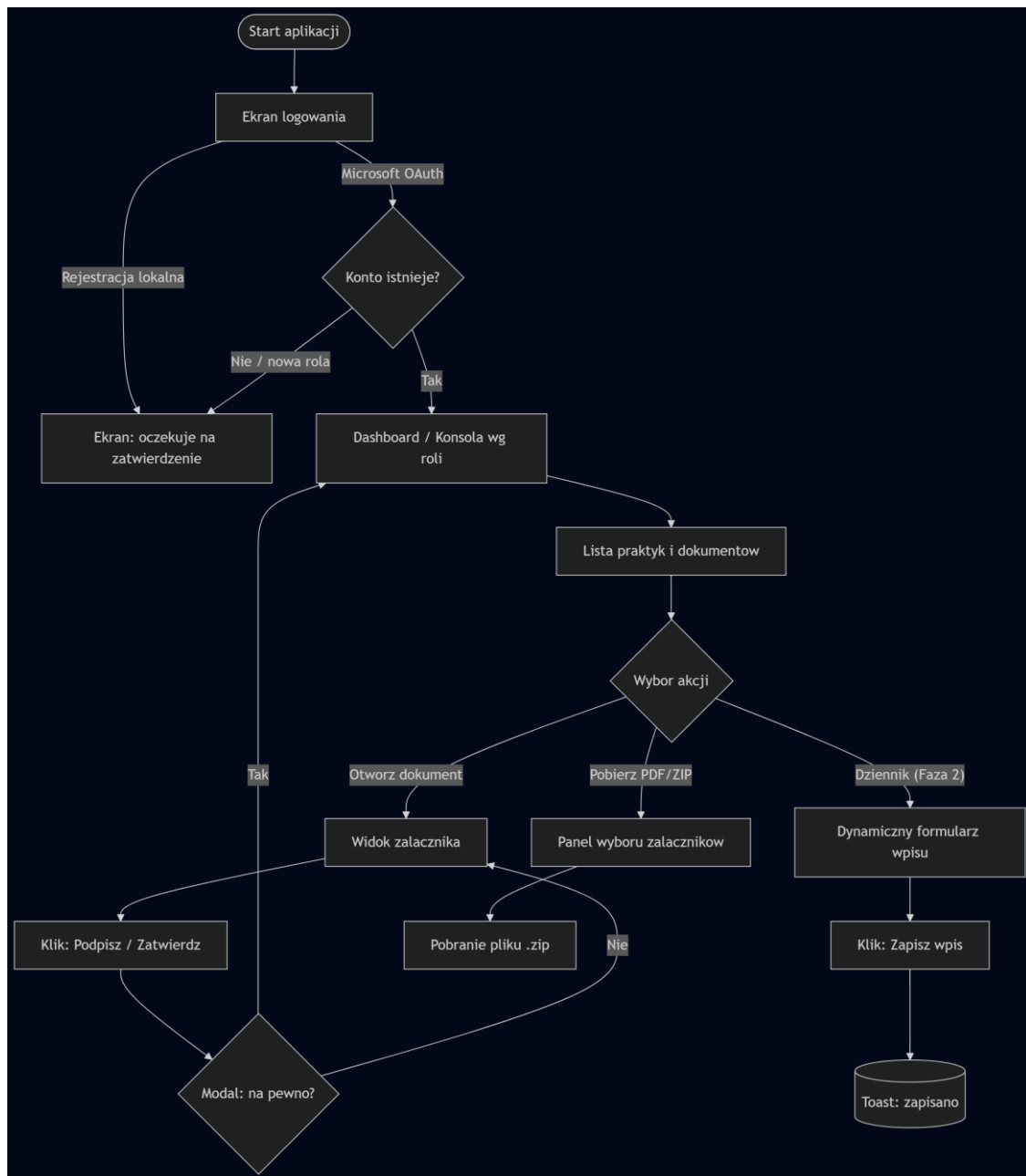
flowchart TD

```
Start([Start aplikacji]) --> Login[Ekran logowania]:::screen
Login -->|Microsoft OAuth| Auth{Konto istnieje?}
Login -->|Rejestracja lokalna| Pending[Ekran: oczekuje na
zatwierdzenie]:::screen
Auth -- Nie / nowa rola --> Pending
Auth -- Tak --> Dash[Dashboard / Konsola wg roli]:::screen

Dash --> Lista[Lista praktyk i dokumentow]:::screen
Lista --> Wybor{Wybor akcji}
Wybor -- "Otworz dokument" --> Podglad[Widok zalacznika]:::screen
Wybor -- "Dziennik (Faza 2)" --> Form[Dynamiczny formularz wpisu]:::screen
Wybor -- "Pobierz PDF/ZIP" --> Zip[Panel wyboru zalacznikow]:::popup

Form --> Zapisz[Klik: Zapisz wpis]
Zapisz --> Toast[(Toast: zapisano)]:::popup
Podglad --> Podpis[Klik: Podpisz / Zatwierdz]
Podpis --> Confirm{Modal: na pewno?}:::popup
Confirm -- Tak --> Dash
Confirm -- Nie --> Podglad
Zip --> Pobierz[Pobranie pliku .zip]
```

Wygenerowany z kodu Mermaid diagram przedstawiono na Rys. 4.:



Rys. 4. Diagram prezentujący interakcje związane z interfejsem

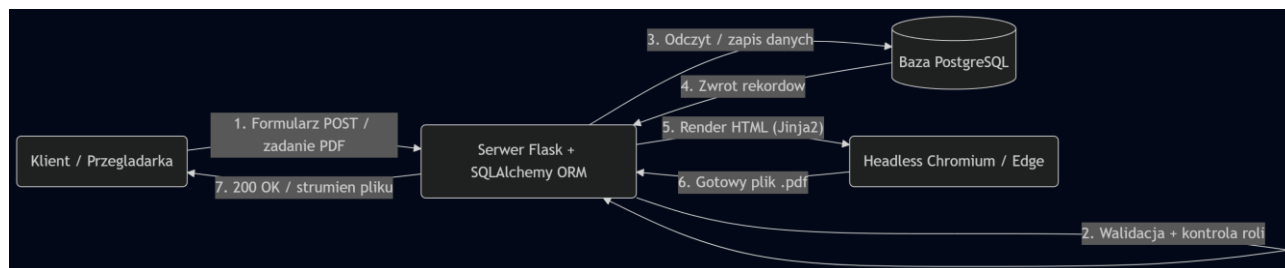
2. Interakcje związane z przepływem danych: Ten diagram obrazuje, jak informacje wędrują pomiędzy warstwą prezentacji (Frontend), serwerem (Backend) a miejscem zapisu (Baza JSON). Kod Mermaid:

flowchart LR

```
Client(Klient / Przeglądarka):::frontend
API(Serwer Flask + SQLAlchemy ORM):::backend
DB[(Baza PostgreSQL)]::database
PDF(Headless Chromium / Edge):::backend

Client -->|"1. Formularz POST / zadanie PDF"| API
API -->|"2. Walidacja + kontrola roli"| API
API -->|"3. Odczyt / zapis danych"| DB
DB -->|"4. Zwrot rekordow"| API
API -->|"5. Render HTML (Jinja2)"| PDF
PDF -->|"6. Gotowy plik .pdf"| API
API -->|"7. 200 OK / strumien pliku"| Client
```

Wygenerowany z kodu Mermaid diagram przedstawiono na Rys. 5.:



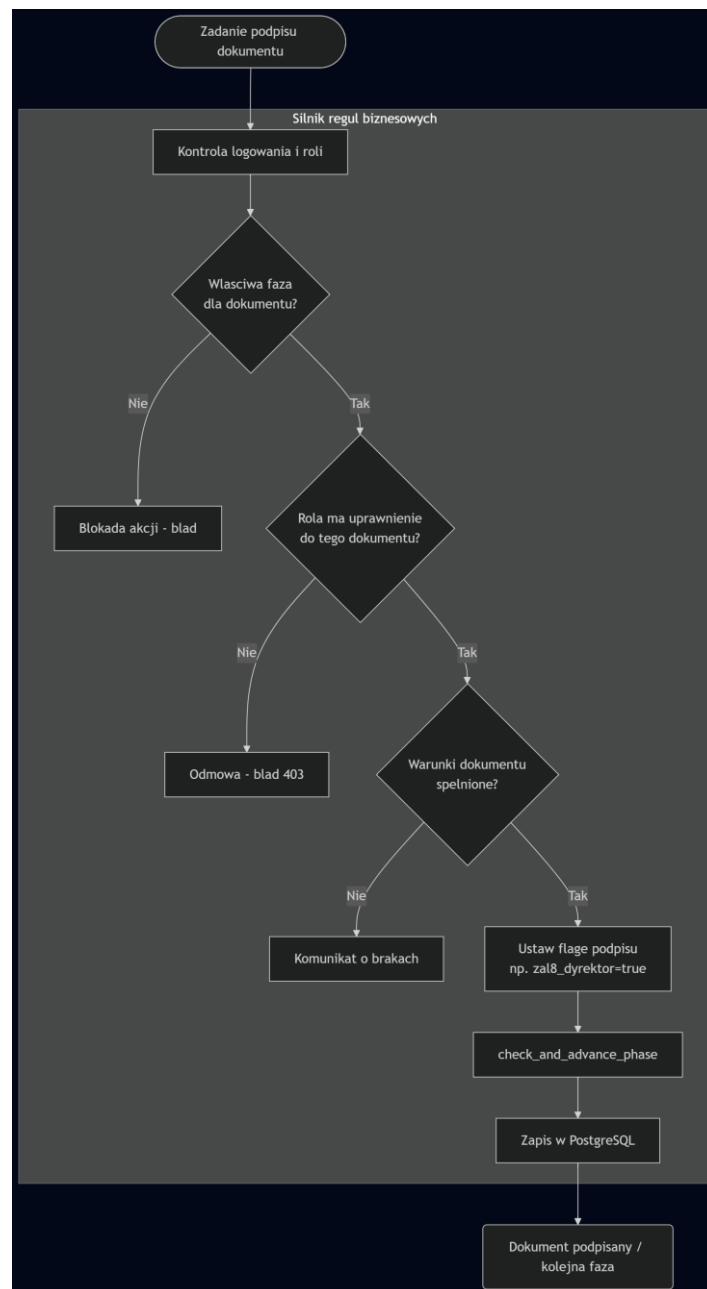
Rys. 5. Diagram przedstawiający interakcje związane z przepływem danych

3. Interakcje związane z logiką biznesową - zakodowane mechanizmy pracy aplikacji, W tym diagramie zwrócono uwagę na ukrytych mechanizmach decyzyjnych aplikacji. Odpowiada on na pytanie: "Jakie warunki muszą zostać spełnione, aby dokument przeszedł do kolejnego etapu?" Kod Mermaid:

flowchart TD

```
Start([Zadanie podpisu dokumentu]) --> Auth[Kontrola logowania i roli]:::action
subgraph Silnik reguł biznesowych
  Auth --> Faza{Wlasciwa faza<br/>dla dokumentu?}:::logic
  Faza -- Nie --> Blok[Blokada akcji - blad]:::action
  Faza -- Tak --> Rola{Rola ma uprawnienie<br/>do tego dokumentu?}:::logic
  Rola -- Nie --> B403[B403[Odmowa - blad 403]]:::action
  Rola -- Tak --> Warunki{Warunki dokumentu<br/>spelnione?}:::logic
  Warunki -- Nie --> Blok2[Komunikat o brakach]:::action
  Warunki -- Tak --> Flaga[Ustaw flage podpisu<br/>np.
zal8_dyrektor=true]:::action
  Flaga --> Advance[check_and_advance_phase]:::action
  Advance --> Commit[Zapis w PostgreSQL]:::action
end
Commit --> Koniec(Dokument podpisany / kolejna faza)
```

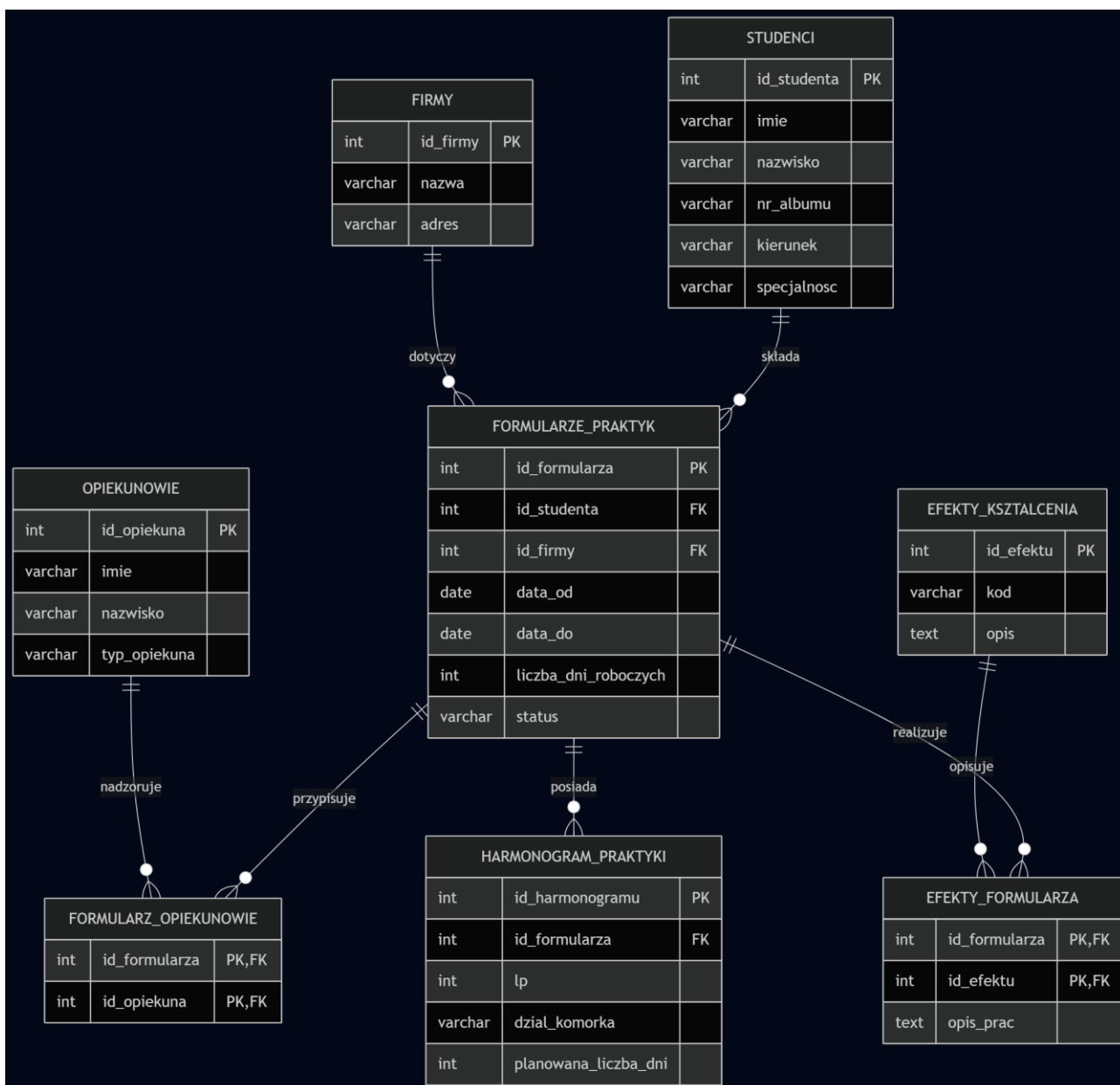
Wygenerowany z kodu Mermaid diagram przedstawiono na Rys. 6.:



Rys. 6. Diagram przedstawiający interakcje związane z logiką biznesową

11. Opracowanie schematu ERD Bazy Danych

Poniższy diagram ERD przedstawia znormalizowaną, podstawową strukturę relacyjnej bazy danych zaprojektowaną na etapie inicjalizacji systemu obsługi praktyk zawodowych. Model ten w wersji bazowej koncentruje się na obsłudze dokumentu startowego, jakim jest Program i Harmonogram praktyki. Struktura bazy została podzielona na encje główne (STUDENCI, FIRMY, OPIEKUNOWIE, FORMULARZE_PRAKTYK), encję szczegółową (HARMONOGRAM_PRAKTYKI) oraz tabele pośrednie i słownikowe (EFEKTY_KSZTALCENIA, EFEKTY_FORMULARZA, FORMULARZ_OPIEKUNOWIE). Taki podział pozwala na całkowite wyeliminowanie redundancji danych i jest zgodny z zasadami normalizacji baz danych. W schemacie precyzyjnie określono klucze główne (PK), klucze obce (FK) oraz krotności relacji – w tym relacje jeden-do-wielu (1:N) oraz relacje wiele-do-wielu (N:M), które zostały poprawnie rozwiązane za pomocą encji pośrednich.



Rys. 7. Schemat ERD bazy danych aplikacji

12. Opracowanie szkieletu tabel w SQL

Na podstawie przygotowanego wcześniej znormalizowanego diagramu ERD, opracowano fizyczny szkielet relacyjnej bazy danych. W strukturze zdefiniowano podstawowe i optymalne typy danych dla każdej z kolumn oraz nałożono sztywne ograniczenia kluczy głównych i kluczy obcych.

W celu zabezpieczenia warstwy danych przed anomaliami oraz wprowadzeniem niepełnych lub błędnych informacji, zaimplementowano reguły takie jak NOT NULL oraz UNIQUE. Dodatkowo wprowadzono ograniczenia sprawdzające typu CHECK realizujące logikę biznesową instytutu (np. wymóg, aby łączna liczba dni roboczych wynosiła dokładnie 120, a data końcowa nie była wcześniejsza niż data rozpoczęcia praktyki). W strukturze tabel uwzględniono także kolumny techniczne i statusowe (status), co umożliwi późniejsze sterowanie cyklem życia dokumentów oraz zarządzanie uprawnieniami użytkowników. Aby zapewnić automatyczne czyszczenie bazy podczas usuwania dokumentów nadrzędnych, w kluczach obcych zastosowano regułę kaskadową ON DELETE CASCADE.

Pełny, wykonywalny kod źródłowy bazy danych w wersji podstawowej został dostarczony w osobnym załączniku o nazwie Szkielet-Bazy-Danych.sql, oraz został przedstawiony poniżej:

```
CREATE TABLE studenci (  
    id_studenta INTEGER PRIMARY KEY AUTOINCREMENT,  
    imie VARCHAR(50) NOT NULL,  
    nazwisko VARCHAR(50) NOT NULL,  
    nr_albumu VARCHAR(20) NOT NULL UNIQUE,  
    kierunek VARCHAR(100) NOT NULL,  
    specjalnosc VARCHAR(100)  
);  
  
CREATE TABLE firmy (  
    id_firmy INTEGER PRIMARY KEY AUTOINCREMENT,  
    nazwa VARCHAR(255) NOT NULL,  
    adres VARCHAR(255)  
);  
  
CREATE TABLE opiekunowie (  
    id_opiekuna INTEGER PRIMARY KEY AUTOINCREMENT,  
    imie VARCHAR(50) NOT NULL,  
    nazwisko VARCHAR(50) NOT NULL,  
    typ_opiekuna VARCHAR(30) NOT NULL CHECK (typ_opiekuna IN ('uczelniany',  
'zakladowy'))  
);  
  
CREATE TABLE formularze_praktyk (  
    id_formularza INTEGER PRIMARY KEY AUTOINCREMENT,  
    id_studenta INTEGER NOT NULL,  
    id_firmy INTEGER NOT NULL,  
    data_od DATE NOT NULL,
```

```

    data_do DATE NOT NULL,
    liczba_dni_roboczych INTEGER NOT NULL CHECK (liczba_dni_roboczych = 120),
    status VARCHAR(30) NOT NULL DEFAULT 'roboczy' CHECK (status IN ('roboczy',
'zatwierdzony', 'odrzucony')),
    FOREIGN KEY (id_studenta) REFERENCES studenci(id_studenta),
    FOREIGN KEY (id_firmy) REFERENCES firmy(id_firmy),
    CHECK (data_do >= data_od)
);

CREATE TABLE formularz_opiekunowie (
    id_formularza INTEGER NOT NULL,
    id_opiekuna INTEGER NOT NULL,
    PRIMARY KEY (id_formularza, id_opiekuna),
    FOREIGN KEY (id_formularza) REFERENCES formularze_praktyk(id_formularza) ON
DELETE CASCADE,
    FOREIGN KEY (id_opiekuna) REFERENCES opiekunowie(id_opiekuna) ON DELETE CASCADE
);

CREATE TABLE efekty_ksztalcenia (
    id_efektu INTEGER PRIMARY KEY AUTOINCREMENT,
    kod VARCHAR(2) NOT NULL UNIQUE,
    opis TEXT NOT NULL
);

CREATE TABLE efekty_formularza (
    id_formularza INTEGER NOT NULL,
    id_efektu INTEGER NOT NULL,
    opis_prac TEXT NOT NULL,
    PRIMARY KEY (id_formularza, id_efektu),
    FOREIGN KEY (id_formularza) REFERENCES formularze_praktyk(id_formularza) ON
DELETE CASCADE,
    FOREIGN KEY (id_efektu) REFERENCES efekty_ksztalcenia(id_efektu) ON DELETE
CASCADE
);

CREATE TABLE harmonogram_praktyki (
    id_harmonogramu INTEGER PRIMARY KEY AUTOINCREMENT,
    id_formularza INTEGER NOT NULL,
    lp INTEGER NOT NULL,
    dzial_komorka VARCHAR(255) NOT NULL,
    planowana_liczba_dni INTEGER NOT NULL CHECK (planowana_liczba_dni > 0),
    FOREIGN KEY (id_formularza) REFERENCES formularze_praktyk(id_formularza) ON
DELETE CASCADE,
    UNIQUE (id_formularza, lp)
);

```

13. Dobór systemu bazy danych

- Do pierwszej wersji projektu wybrano system SQLite. Jest to standardowa biblioteka w języku Python, która nie wymaga instalacji oraz konfiguracji osobnego serwera. SQLite przechowuje całą bazę w jednym lekkim pliku, co pozwala na błyskawiczne opracowanie modelu danych i łatwe testowanie prototypu. Było to szczególnie wygodne na wczesnym etapie, gdy najpierw powstawała podstawowa struktura bazy, a następnie była ona stopniowo rozbudowywana o kolejne tabele i dokumenty.
- Głównym ograniczeniem SQLite jest brak wsparcia dla wysokiej współbieżności zapisu (blokowanie całego pliku bazy) oraz ograniczone możliwości zarządzania prawami użytkowników na poziomie samego silnika. Z tego powodu jako silnik docelowy wybrano PostgreSQL — wydajny, w pełni relacyjny system bazodanowy o otwartym kodzie, obsługujący współbieżny dostęp wielu użytkowników, transakcje oraz zaawansowane typy danych i więzy integralności.
- Po ustabilizowaniu modelu danych projekt zostanie przeniesiony z SQLite na PostgreSQL, który będzie silnikiem wykorzystywanym w docelowej wersji aplikacji (połączenie konfigurowane przez zmienną DATABASE_URL). SQLite pozostanie natomiast wygodną opcją do lokalnego dewelopmentu (brak problemów z uruchomieniem środowiska).

14. Dobór narzędzia do prototypowania bazy danych

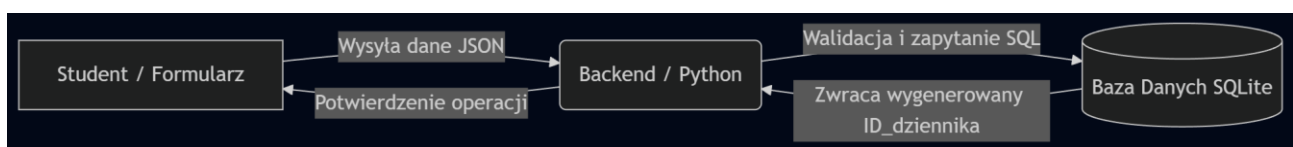
Do prototypowania wybrano narzędzie DBeaver Community. Jest to uniwersalne, darmowe środowisko graficzne, które obsługuje wiele systemów baz danych, co czyni go idealną i lżejszą alternatywą dla MySQL Workbench. Dzięki obsłudze wielu sterowników sprawdzi się doskonale zarówno na obecnym etapie prototypowania (SQLite), jak i w przyszłości, po ewentualnej migracji projektu na docelowy serwer MariaDB lub PostgreSQL.

Zastosowanie w projekcie (w kontekście aplikacji do obsługi praktyk): W ramach realizowanego projektu systemu obsługi praktyk zawodowych, DBeaver znajduje następujące zastosowanie:

- Inicjalizacja prototypu: Posłuży do fizycznego utworzenia pliku bazy SQLite i wykonania na nim skryptu SQL (DDL) z definicjami tabel, kluczami i relacjami.
- Wizualizacja i walidacja struktury: Posłuży do automatycznego wygenerowania fizycznego diagramu ERD na podstawie wdrożonych tabel, co pozwala na bieżąco weryfikować poprawność kluczy obcych i powiązań pomiędzy użytkownikami a dziennikami praktyk.
- Zarządzanie danymi testowymi: Będzie wykorzystywany jako główne narzędzie GUI do ręcznego "zasilenia" bazy początkowymi danymi słownikowymi (np. dodanie ról Student, Opiekun_Uczelniany) oraz do testowania poprawności zapisów wysyłanych przez aplikację z poziomu kodu backendowego.

15. Propozycja narzędzia do organizacji projektu bazy danych

Do dokumentowania i współdzielenia schematu zaproponowano narzędzie Mermaid. Pozwala ono na modelowanie diagramów (w tym ERD oraz przepływów) za pomocą zwykłego tekstu. Ułatwia to wersjonowanie struktury bazy danych w repozytorium GitHub, ponieważ każdą zmianę w schemacie widać bezpośrednio w plikach tekstowych (np. w pliku README.md), z pominięciem konieczności załączania plików binarnych programów graficznych.



Rys. 8. Wizualizacja przepływu danych

16. Opracowanie diagramu ERD dla formularza praktyki oraz wskazanie kluczy głównych i obcych

Diagram ERD uwzględniający strukturę formularza, słowników oraz harmonogramu, wraz ze ścisłym wskazaniem kluczy głównych (PK) i obcych (FK), został opracowany i zaprezentowany we wcześniejszym rozdziale dokumentacji 11. Opracowanie schematu ERD Bazy Danych.

Wyjaśnienie rozdzielania danych na kilka tabel: Dane zostały poddane normalizacji i rozdzielone na relacyjne encje (np. wydzielenie studentów, firm i opiekunów do osobnych tabel słownikowych), aby wyeliminować zjawisko redundancji (powtarzania się danych). Trzymanie wszystkiego w jednej płaskiej tabeli powodowałoby anomalie modyfikacji (np. zmiana adresu firmy wymagałaby aktualizacji tysięcy rekordów) oraz uniemożliwiałoby łatwe przeszukiwanie bazy pod kątem konkretnych efektów kształcenia.

Utworzenie tabel w SQL: Skrypt DDL realizujący strukturę tabel, zawierający niezbędne klucze oraz ograniczenia typu CHECK i UNIQUE:

```
CREATE TABLE studenci (
    id_studenta INTEGER PRIMARY KEY AUTOINCREMENT,
    imie VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL,
    nr_albumu VARCHAR(20) NOT NULL UNIQUE,
    kierunek VARCHAR(100) NOT NULL,
    specjalnosc VARCHAR(100)
);

CREATE TABLE firmy (
```

```

    id_firmy INTEGER PRIMARY KEY AUTOINCREMENT,
    nazwa VARCHAR(255) NOT NULL,
    adres VARCHAR(255)
);

CREATE TABLE opiekunowie (
    id_opiekuna INTEGER PRIMARY KEY AUTOINCREMENT,
    imie VARCHAR(50) NOT NULL,
    nazwisko VARCHAR(50) NOT NULL,
    typ_opiekuna VARCHAR(30) NOT NULL CHECK (typ_opiekuna IN ('uczelniany',
'zakladowy'))
);

CREATE TABLE formularze_praktyk (
    id_formularza INTEGER PRIMARY KEY AUTOINCREMENT,
    id_studenta INTEGER NOT NULL,
    id_firmy INTEGER NOT NULL,
    data_od DATE NOT NULL,
    data_do DATE NOT NULL,
    liczba_dni_roboczych INTEGER NOT NULL CHECK (liczba_dni_roboczych = 120),
    status VARCHAR(30) NOT NULL DEFAULT 'roboczy' CHECK (status IN ('roboczy',
'zatwierdzony', 'odrzucony')),
    FOREIGN KEY (id_studenta) REFERENCES studenci(id_studenta),
    FOREIGN KEY (id_firmy) REFERENCES firmy(id_firmy),
    CHECK (data_do >= data_od)
);

CREATE TABLE formularz_opiekunowie (
    id_formularza INTEGER NOT NULL,
    id_opiekuna INTEGER NOT NULL,
    PRIMARY KEY (id_formularza, id_opiekuna),
    FOREIGN KEY (id_formularza) REFERENCES formularze_praktyk(id_formularza) ON
DELETE CASCADE,
    FOREIGN KEY (id_opiekuna) REFERENCES opiekunowie(id_opiekuna) ON DELETE CASCADE
);

CREATE TABLE efekty_ksztalcenia (
    id_efektu INTEGER PRIMARY KEY AUTOINCREMENT,
    kod VARCHAR(2) NOT NULL UNIQUE,
    opis TEXT NOT NULL
);

CREATE TABLE efekty_formularza (
    id_formularza INTEGER NOT NULL,
    id_efektu INTEGER NOT NULL,
    opis_prac TEXT NOT NULL,
    PRIMARY KEY (id_formularza, id_efektu),

```

```

    FOREIGN KEY (id_formularza) REFERENCES formularze_praktyk(id_formularza) ON
DELETE CASCADE,
    FOREIGN KEY (id_efektu) REFERENCES efekty_ksztalcenia(id_efektu) ON DELETE
CASCADE
);

CREATE TABLE harmonogram_praktyki (
    id_harmonogramu INTEGER PRIMARY KEY AUTOINCREMENT,
    id_formularza INTEGER NOT NULL,
    lp INTEGER NOT NULL,
    dzial_komorka VARCHAR(255) NOT NULL,
    planowana_liczba_dni INTEGER NOT NULL CHECK (planowana_liczba_dni > 0),
    FOREIGN KEY (id_formularza) REFERENCES formularze_praktyk(id_formularza) ON
DELETE CASCADE,
    UNIQUE (id_formularza, lp)
);

```

Przygotowanie poprawnych danych testowych: Poniższe polecenia INSERT wprowadzają do bazy zbiór prawidłowych danych początkowych:

```

INSERT INTO studenci (id_studenta, imie, nazwisko, nr_albumu, kierunek,
specjalnosc)
VALUES (1, 'Jan', 'Kowalski', '123456', 'Informatyka', 'Aplikacje internetowe');

INSERT INTO firmy (id_firmy, nazwa, adres)
VALUES (1, 'ABC Software Sp. z o.o.', 'Elblag');

INSERT INTO formularze_praktyk (id_formularza, id_studenta, id_firmy, data_od,
data_do, liczba_dni_roboczych)
VALUES (1, 1, 1, '2026-07-01', '2026-12-15', 120);

INSERT INTO opiekunowie (id_opiekuna, imie, nazwisko, typ_opiekuna)
VALUES (1, 'Anna', 'Nowak', 'uczelniany'), (2, 'Piotr', 'Zielinski', 'zakladowy');

INSERT INTO formularz_opiekunowie (id_formularza, id_opiekuna)
VALUES (1, 1), (1, 2);

```

Przygotowanie błędnych danych testowych dla przypadków brzegowych: Próba wprowadzenia poniższych danych wywoła błędy na poziomie silnika bazy danych, blokując niepoprawne akcje:

```
-- Błąd: Naruszenie UNIQUE (duplikat numeru albumu)
INSERT INTO studenci (id_studenta, imie, nazwisko, nr_albumu, kierunek,
specjalnosc)
VALUES (2, 'Adam', 'Nowicki', '123456', 'Informatyka', 'Sieci komputerowe');

-- Błąd: Naruszenie CHECK (liczba dni różna od 120)
INSERT INTO formularze_praktyk (id_formularza, id_studenta, id_firmy, data_od,
data_do, liczba_dni_roboczych)
VALUES (2, 1, 1, '2026-07-01', '2026-09-30', 80);

-- Błąd: Naruszenie CHECK (data zakończenia wcześniejsza niż data rozpoczęcia)
INSERT INTO formularze_praktyk (id_formularza, id_studenta, id_firmy, data_od,
data_do, liczba_dni_roboczych)
VALUES (3, 1, 1, '2026-10-01', '2026-07-01', 120);
```

Uruchomienie zapytań kontrolnych oraz opis, które zapytania wykrywają błędy:

Zdefiniowano zestaw zapytań SELECT, które pełnią rolę audytorów poprawności (jeśli zwrócą jakikolwiek rekord, oznacza to znalezienie błędu w formularzu):

Wykrywanie braków formalnych: Sprawdza, czy w formularzu nie brakuje przypisania firmy, studenta lub zdefiniowanych dat .

```
SELECT * FROM formularze_praktyk WHERE id_studenta IS NULL OR id_firmy IS NULL OR
data_od IS NULL OR data_do IS NULL OR liczba_dni_roboczych IS NULL;
```

Wykrywanie błędów logicznych harmonogramu: Waliduje, czy zadeklarowana liczba dni wynosi dokładnie 120 i czy data zakończenia jest chronologiczna .

```
SELECT id_formularza FROM formularze_praktyk WHERE liczba_dni_roboczych <> 120 OR
data_do < data_od;
```

Wykrywanie braków w efektach kształcenia: Weryfikuje, czy formularz posiada przypisane dokładnie 13 efektów .

```
SELECT id_formularza FROM efekty_formularza GROUP BY id_formularza HAVING COUNT(*)
<> 13;
```

Wykrywanie błędów w bilansie dni (Harmonogram): Zlicza przypisane dni w poszczególnych komórkach zakładu i alarmuje, jeśli ich suma nie domyka się w liczbie 120 .

```
SELECT id_formularza FROM harmonogram_praktyki GROUP BY id_formularza HAVING
SUM(planowana_liczba_dni) <> 120;
```

17. Rozszerzony model ERD (Postgres)

25

Opis powiązań między formularzami

Architektura opiera się na jednej encji centralnej formularze_praktyk. Wszystkie pozostałe dokumenty łączą się z nią przez klucz obcy id_formularza:

Dokument	Tabela	Krotność
Program i harmonogram (Zał. 2a)	harmonogram_praktyki, efekty_formularza	1:N
Karta praktyki (Zał. 3.1/3.2)	karty_praktyki	1:1
Potwierdzenie efektów (Zał. 4)	potwierdzenia_efektow	1:N (13)
Dziennik (Zał. 6)	dzienniki_praktyk → wpisy_dziennika	1:1 → 1:N
Sprawozdanie (Zał. 7)	sprawozdania	1:1
Ankieta (Zał. 5)	ankiety	1:1
Protokół (Zał. 8)	protokoly_zaliczenia	1:1

Dane studenta i okres praktyki (data_od, data_do) przechowywane są wyłącznie w tabeli formularze_praktyk (+ słownik studenci). Żaden dokument podrzędny nie duplikuje tych pól, dziedziczy kontekst przez id_formularza.

Dzięki temu rozbieżność danych studenta lub okresu między formularzami jest fizycznie niemożliwa. Efekty kształcenia pochodzą z jednego słownika efekty_ksztalcenia (01–13), wspólnego dla tabel efekty_formularza, potwierdzenia_efektow i wpisy_dziennika.

Zestaw reguł walidacyjnych dla każdego dokumentu

- Karta praktyki: pola obowiązkowe: id_formularza, oceny; reguły: oceny parametryczne w zakresie 2–5, obecność daty BHP i podpisu. SQL: kontrola NULL, CHECK (ocena BETWEEN 2 AND 5), porównanie istnienia rekordu.
- Potwierdzenie efektów: dokładnie 13 pozycji, każda z decyzją czy_uzyskany ∈ {0,1} (typ wyklucza jednocześnie „tak/nie”). SQL: GROUP BY id_formularza HAVING COUNT(*) <> 13.
- Dziennik: 120 wpisów, każdy z niepustym opisem i przypisanym efektem, daty w okresie i tylko dni robocze. SQL: zliczanie wpisów, kontrola NULL/pustych, porównanie dat z formularze_praktyk.
- Sprawozdanie: obecność 3 sekcji (charakterystyka_miejsca, opis_i_analiza_prac, samoocena_kompetencji), wymuszona NOT NULL. SQL: kontrola NULL/pustych ciągów.

Zestaw reguł walidacji między dokumentami

- Ten sam student / zgodność okresu — gwarantowane konstrukcją (jedna encja centralna, brak duplikacji).
- 120 dni: $\text{SUM}(\text{harmonogram_praktyki.planowana_liczba_dni}) = 120$ oraz $\text{COUNT}(\text{wpisy_dziennika}) = 120$.
- Spójność 13 efektów: zgodność zbiorów między efekty_formularza, potwierdzenia_efektow i efektami użytymi we wpisy_dziennika.
- Pokrycie okresu: każdy data_wpisu mieści się w data_od..data_do.

Propozycja zapytań SQL

Zdefiniowano analityczne i walidacyjne zapytania z użyciem poleceń SELECT i JOIN:

- Zapytanie wykrywające konflikty dat: Skrypt łączący tabele wpisów z formularzem nadrzędnym, odrzucający za pomocą klauzuli WHERE te krotki, których data_wpisu wykracza poza przedział data_od i data_do.
- Zapytanie weryfikujące kompletność efektów: Zapytanie grupujące (GROUP BY id_formularza) i zliczające rekordy w tabeli Potwierdzeń Efektów, wyświetlające tylko te dokumenty, gdzie wynik jest mniejszy lub większy niż przewidziane regulaminowo 13 .
- Zapytanie wykrywające rozbieżności ilościowe: Porównanie między tabelami pobierające zagregowaną listę wpisów z danego dziennika praktyk z polem całkowitej liczby zaplanowanych godzin.

Przykładowe zapytania kontrolne (zwracają błędne rekordy)

```
-- Dziennik nie ma 120 wpisów
SELECT id_formularza FROM dzienniki_praktyk d
JOIN wpisy_dziennika w ON w.id_dziennika = d.id_dziennika
GROUP BY id_formularza HAVING COUNT(*) <> 120;
```

```
-- Potwierdzenia efektów nie obejmują dokładnie 13 pozycji
SELECT id_formularza FROM potwierdzenia_efektow
GROUP BY id_formularza HAVING COUNT(*) <> 13;
```

```
-- Wpis dziennika poza okresem praktyki
SELECT w.id_wpisu FROM wpisy_dziennika w
JOIN dzienniki_praktyk d ON d.id_dziennika = w.id_dziennika
JOIN formularze_praktyk f ON f.id_formularza = d.id_formularza
WHERE w.data_wpisu < f.data_od OR w.data_wpisu > f.data_do;
```

Analiza przypadków brzegowych

Zapytania pełnią rolę audytu — jeśli zwrócą jakikolwiek rekord, oznacza to wykryty błąd.

1. Różne daty praktyki w różnych formularzach Zablokowane konstrukcją: daty (data_od, data_do) istnieją wyłącznie w encji centralnej, dokumenty podrzędne ich nie kopią — rozbieżność jest niemożliwa.

Pozostaje kontrola poprawności samego przedziału:

```
SELECT id_formularza FROM formularze_praktyk WHERE data_do < data_od;
```

2. Brak wpisów w dzienniku

```
SELECT d.id_formularza
FROM dzienniki_praktyk d
LEFT JOIN wpisy_dziennika w ON w.id_dziennika = d.id_dziennika
WHERE w.id_wpisu IS NULL;
```

3. Brak efektów w jednym z dokumentów Efekty występują w trzech miejscach, każde musi obejmować 13 pozycji:

```
-- Zał. 2a: niepełna deklaracja efektów
SELECT id_formularza FROM efekty_formularza
GROUP BY id_formularza HAVING COUNT(*) <> 13;

-- Zał. 4: niepełne potwierdzenia
SELECT id_formularza FROM potwierdzenia_efektow
GROUP BY id_formularza HAVING COUNT(*) <> 13;

-- Formularz w fazie >=3 bez ani jednego potwierdzenia
SELECT f.id_formularza FROM formularze_praktyk f
LEFT JOIN potwierdzenia_efektow p ON p.id_formularza = f.id_formularza
WHERE f.faza_procesu >= 3 AND p.id_potwierdzenia IS NULL;
```

4. Niespójne dane studenta Zablokowane konstrukcją (jeden rekord studenta współdzielony przez wszystkie formularze). Kontrola integralności, formularz bez studenta lub student z brakami:

```
SELECT f.id_formularza
FROM formularze_praktyk f
LEFT JOIN studenci s ON s.id_studenta = f.id_studenta
WHERE s.id_studenta IS NULL
    OR s.imie IS NULL OR s.nazwisko IS NULL OR s.nr_albumu IS NULL;
```

5. Ocena pozytywna przy braku efektów Karta ma ocenę zaliczającą (≥3), a któryś efekt nie został uzyskany:

```

SELECT k.id_formularza
FROM karty_praktyki k
WHERE CAST(NULLIF(k.ocena_zopz_param, '') AS NUMERIC) >= 3
      AND EXISTS (
          SELECT 1 FROM potwierdzenia_efektow p
          WHERE p.id_formularza = k.id_formularza AND p.czy_uzyskany = 0
      );

```

6. Brak podpisów Praktyka oznaczona jako zakończona (usos_wpisany), a brakuje któregośkolwiek wymaganego podpisu:

```

SELECT id_formularza
FROM formularze_praktyk
WHERE usos_wpisany = TRUE
      AND (zal1_zopz = FALSE OR zal1_dyrektor = FALSE
          OR zal2a_uopz = FALSE OR zal31_dyrektor = FALSE
          OR dziennik_zatwierdzony = FALSE
          OR zal3_strona2_zopz = FALSE OR zal3_strona2_uopz = FALSE
          OR zal4_uopz = FALSE OR zal7_zopz = FALSE
          OR zal8_dyrektor = FALSE OR zal8_uopz = FALSE);

```

7. Sprzeczne informacje (praktyka zakończona, brak potwierdzenia):

```

-- Zakończona, ale brak protokołu zaliczenia (Zał. 8)
SELECT f.id_formularza
FROM formularze_praktyk f
LEFT JOIN protokoly_zaliczenia pr ON pr.id_formularza = f.id_formularza
WHERE f.usos_wpisany = TRUE AND pr.id IS NULL;

-- Zakończona, ale brak daty szkolenia BHP na Karcie
SELECT f.id_formularza
FROM formularze_praktyk f
JOIN karty_praktyki k ON k.id_formularza = f.id_formularza
WHERE f.usos_wpisany = TRUE AND k.data_szkolenia_bhp IS NULL;

```